CSCE 2303 Project: A Cortex-M0 Instruction Set Simulator

1. Introduction

You will write an instruction set simulator for the Arm Cortex-M0 and use it to run real compiled bare-metal C and assembly programs. The point is RISC-V: you only really understand a design once you have seen the alternative, and the M0's instruction set, Thumb, made different choices almost everywhere. It has condition flags instead of compare-and-branch, 16-bit instructions that mostly reach only eight registers, no zero register, push and pop with register lists, and constants loaded from memory.

You will hand in four things: the simulator, your own test programs, a short study comparing Thumb with RV32I, and yourselves, for an interview.

What an ISS is

An instruction set simulator (ISS) is a program that executes another processor's machine code. It keeps that processor's registers and memory in ordinary variables and arrays and runs one loop: fetch the instruction at pc, decode it, do what it says, move pc on. It reproduces everything a programmer can see, the values in registers and memory, but not how the hardware gets there: no pipeline, no gates, no clock. RARS is an ISS for RISC-V. You will write one for the M0.

The Cortex-M0

The Cortex-M0 is Arm's smallest 32-bit core, made for cheap, battery-powered microcontrollers. It implements the ARMv6-M architecture, which has only the Thumb instruction set: almost every instruction is 16 bits, and there is no ARM mode. The Cortex-M0+ runs the same instructions with a shorter pipeline; the RP2040 on the Raspberry Pi Pico has two of them. Your simulator models the registers, flags, memory and the 43 instructions in section 4, with no interrupts or peripherals.

Tools and languages

Use Compiler Explorer to build and link both Thumb assembly and freestanding C. The supplied C runtime contains only a vector table, minimal startup and direct SVC wrappers; no standard library, libgcc or operating system is linked. Both templates produce readelf section dumps for your simulator and refiss. Build the simulator itself with your normal host C/C++ compiler or Python 3.

refiss is our reference simulator, an implementation of this handout built on a patched rp2040js Cortex-M0 core and cross-checked against Unicorn. It is a single web page that you download from Canvas and open in your browser; section 7 explains how to use it. Use it to run your Thumb programs before your simulator works, and compare traces with it when you debug. Every expected test result comes from it. Treat it as a black box: your simulator may not use rp2040js or any other emulator.

Write your simulator in C, C++ or Python 3. No other language is accepted.

2. Teams and schedule

Work in teams of three, and every member contributes code to every checkpoint (see Who does what below). The project starts on Tuesday, November 3, 2026 and runs for four weeks. We grade the commit tagged cp1, cp2, cp3 or final before its deadline. All deadlines below are at 23:59 Cairo time (Africa/Cairo).

Checkpoint	Due	What must work	How we check it
CP1: Learn the ISA	Tue 10 Nov 2026	Three of your RARS homework programs rewritten in Thumb assembly, committed with their dumps. In your simulator: the loader and a decoder for all 43 instructions, shown through `--disasm`.	We run your three programs on `refiss` and compare their output with your RARS versions' on the same input. We run `iss --disasm` on our test dumps and compare it line by line with objdump's disassembly of the same programs (section 6).
CP2: Run your own programs	Tue 17 Nov 2026	Reset and the fetch loop, arithmetic, logic and flags, all branches, BL and BX, word loads and stores, PUSH and POP, the console services, the trace.	Your CP1 programs on your simulator. Public CP2 tests, trace line by trace line.
CP3: The full set	Tue 24 Nov 2026	The rest of section 4, all faults, all Core services.	Bare-metal C programs using direct SVC wrappers. Hidden tests.
Final	Tue 1 Dec 2026	Extensions, the comparison study, the interview.	Sections 8 to 10.

Write Thumb by hand from day one, before the simulator; teams that do build much better decoders. Your simulator must build and run on Linux with the command line in section 6.

Canvas has the Compiler Explorer templates, `refiss`, the public tests and the starter skeletons (section 13). Section 12 maps RV32I onto Thumb; keep it open while you work.

Who does what

Each of you owns one part of the simulator and writes its code, under your own name in the repository history. A sensible split:

Member	Owns
A	The decoder and disassembler (CP1), then loads, stores, LDM, STM, PUSH and POP
B	Arithmetic, logic, shifts, flags and branches, including BL, BX and BLX
C	The loader, memory map, reset, faults, SVC services, trace and command line

Each member also writes one of the three CP1 Thumb programs, at least four of the team's test programs and some of the extension code. Review each other's code before every tag: at the interview each of you answers for all of it, not only your own part.

Every checkpoint tag includes a file `CONTRIBUTIONS.md` with a few lines per member saying what they wrote. We check it against the commit history. A member whose contribution to a checkpoint is missing or trivial gets no marks for that checkpoint.

Submit your repository URL and the checkpoint tag through the corresponding Canvas assignment. Ensure the instructor and TAs can access the repository. Each tag must include source code, build/run instructions, tests and CONTRIBUTIONS.md; final also includes the team comparison report and a separately named reflection for each member.

3. The machine

The machine is a Cortex-M0 reduced to its programmer's model: sixteen registers, four flags, two memory regions and no interrupts. The registers are r0 to r12, sp (r13), lr (r14) and pc (r15), and the flags N, Z, C and V live in the APSR. There is one stack pointer, no privilege levels and no ARM state.

Compared with RV32I there are 16 registers instead of 32, and none is wired to zero. pc is r15: reading it gives the current address plus 4, and writing it is a branch. The flags are state RISC-V does not have: a compare sets them and a later branch tests them.

Region	Start address	Size	Access
ROM	`0x0000_0000`	1 MB	Read and execute. Writes fault.
RAM	`0x2000_0000`	256 KB	Read, write and execute.

Every other address is unmapped. Memory is little-endian; word accesses must be 4-byte aligned and halfword accesses 2-byte aligned.

At reset, sp is loaded from the word at address 0 and pc from the word at address 4, as on the real M0. The stack pointer is always a multiple of 4: the reset value and every write to sp, such as mov sp, r0, have bits 1 and 0 cleared. Bit 0 of the reset vector must be 1, marking a Thumb address; clear it to get pc. lr starts at 0xFFFFFFFF and everything else at 0. ROM and RAM bytes not supplied by the dump start at zero. A RISC-V core instead starts at a fixed address and its start-up code sets sp, which is why every M0 image begins with the vector table.

There is no exception model. On a fault, print `FAULT <type> at pc=0x%08x` to stderr and exit with status 255, much as RARS stops on a misaligned `lw` when no trap handler is installed.

For a reset INVSTATE fault, print the original reset-vector value as pc. For an instruction fault, print the address of that instruction; a failed instruction fetch reports the attempted fetch address. A faulting instruction does not retire. An access that is misaligned and also unmapped, or misaligned and also a write to ROM, raises UNALIGNED.

Fault	Raised when
UNDEFINED	An encoding outside our subset, including M0 instructions we left out, or an unknown SVC number.
UNALIGNED	A word or halfword load or store to a misaligned address.
BUS	An access to an unmapped address, a write to ROM, or an SVC pointer outside mapped memory.
INVSTATE	BX, BLX or POP {pc} to an address with bit 0 clear, or such a reset vector.

4. The instruction set

You implement the 43 instructions below, including the encodings used by the supplied test programs and the course build configuration. Instructions outside this subset must raise UNDEFINED, even when they are legal on a real M0.

Group	Instructions
Move	MOV, MVN
Arithmetic	ADD, ADR, ADC, SUB, SBC, RSB (NEG), MUL
Logic and shifts	AND, ORR, EOR, BIC, TST, LSL, LSR, ASR, ROR
Compare	CMP, CMN
Extend and reverse	SXTB, SXTH, UXTB, UXTH, REV
Load and store	LDR, LDRB, LDRH, LDRSB, LDRSH, STR, STRB, STRH
Multiple	LDM, STM, PUSH, POP
Control	B (including conditional B), BL, BX, BLX (register)
System	SVC, NOP

Each name covers all of its ARMv6-M encodings, and the assembler picks which one a line becomes; ADD alone has six. The ARMv6-M Architecture Reference Manual is your specification. The same 43-instruction subset applies to hand-written assembly and compiler-generated C.

These legal M0 instructions are left out and must raise UNDEFINED: BKPT, CPSID, CPSIE, MRS, MSR, DMB, DSB, ISB, WFI, WFE, SEV, YIELD, UDF, REV16 and REVSH.

ADR forms an address as Align(current instruction address + 4, 4) + (imm8 << 2), writes it to a low register and preserves all flags. It does not read memory. Objdump may display its encoding as add rD, pc, #offset.

Addressing modes

RV32I has one addressing mode, a base register plus a signed 12-bit offset. The M0 has five:

Mode	Example	Instructions	Offset	RV32I
Register + immediate	`ldr r0, [r1, #8]`	LDR, STR, LDRH, STRH, LDRB, STRB	0 to 124 for words, 0 to 62 for halfwords, 0 to 31 for bytes; unsigned and scaled	`lw a0, 8(a1)`
Register + register	`ldr r0, [r1, r2]`	Every load and store; the only form LDRSB and LDRSH have	None	Needs an `add` first
sp + immediate	`ldr r0, [sp, #16]`	LDR, STR	0 to 1020, words only	`lw a0, 16(sp)`
pc-relative	`ldr r0, [pc, #36]`, usually written `ldr r0, =value`	LDR	0 to 1020, forward only, from pc rounded down to a multiple of 4	`auipc` then `lw`
Register list	`ldm r0!, {r1-r3}`, `push {r4, lr}`	LDM, STM, PUSH, POP	Consecutive words; the base register is updated	One `lw` or `sw` per register

There are no negative offsets and no pre- or post-indexing; those came with Thumb-2. Use a SUB first, or register + register with a negative index. Other immediates are small too: 3 bits in `adds r0, r1, #7`, 8 bits in MOVS, CMP and `adds r0, #200`, 5 bits for shifts and 7 bits scaled by 4 for sp, against 12 bits almost everywhere on RV32I.

What people get wrong

Read the manual on each of these before you write the code.

1. Reading pc gives the current address plus 4, and literal loads round that down to a multiple of 4. On RISC-V, auipc gives its own address.
2. Only BL is 32 bits. A first halfword with bits 15 to 11 equal to `11101`, `11110` or `11111` starts a 32-bit instruction, and anything but BL is UNDEFINED. In BL's offset, I1 = NOT(J1 XOR S) and I2 = NOT(J2 XOR S); most people miss the inversion.
3. Add, subtract, compare and negate set N, Z, C and V. The logical forms, TST and MOVS with an immediate set N and Z. Shifts set N, Z and C. MULS sets N and Z. ADD and MOV with a high register, ADD and SUB on sp, ADR, the extends, REV, loads, stores and branches leave the flags alone.
4. On subtraction C means "no borrow": after `cmp r0, r1`, C is 1 when r0 ≥ r1 unsigned, the opposite of sltu. SBC computes Rn − Rm − NOT(C).
5. An immediate shift of 0 means 32 for LSRS and ASRS. LSLS with 0 is MOVS and leaves C unchanged.
6. Register shifts use only the bottom byte of Rm; 0 leaves the value and C unchanged. Amounts of 32 and above have exact rules: look them up rather than trusting your language's `<<`.
7. PUSH stores the lowest-numbered register at the lowest address. POP {pc} is a branch and needs bit 0 set.
8. LDM writes back unless the base register is in the list. STM always writes back.
9. BL and BLX set lr to the return address with bit 0 set, so BX lr returns; together they are the M0's jal ra and ret.
10. MOV pc, Rm and ADD pc, Rm are plain branches; bit 0 is ignored.
11. In a conditional branch, condition `1110` is permanently undefined and `1111` is SVC.

Encodings the manual calls UNPREDICTABLE never appear in our tests. Do anything sensible with them, but do not crash.

5. SVC services

`svc #n` does what `ecall` does in RARS: the simulator performs a service and the program carries on. In RARS the service number is a run-time value in a7; on the M0 it is a constant inside the instruction. Arguments start in r0 instead of a0.

```
# RV32I in RARS: print 42, then exit with status 0
        li      a0, 42
        li      a7, 1             # PrintInt
        ecall
        li      a0, 0
        li      a7, 93            # Exit2
        ecall

@ Thumb on your simulator: the same program
        movs    r0, #42
        svc     #0x12             @ PUTINT
        movs    r0, #0
        svc     #0x00             @ EXIT
```

On real hardware both instructions trap to an operating system, RISC-V through mtvec and the M0 through its SVCall exception. RARS and your simulator skip the trap and do the work directly.

- The service number is the instruction's 8-bit immediate.
- Arguments are in r0 to r3. The result is in r0, or r0 (low) and r1 (high) for 64 bits. Every other register and the flags are preserved.

- File descriptors 0, 1 and 2 are stdin, stdout and stderr, and the only ones. READ accepts only descriptor 0 and WRITE only 1 and 2; any other returns -9 (bad descriptor).
- A pointer that reaches outside mapped memory, or a buffer that would be written into ROM, is a BUS fault at the SVC.
- An unknown service number is an UNDEFINED fault.

EXIT and the console services 0x10 to 0x15 are due at CP2. The remaining Core services are due at CP3. Extended services belong to the extensions in section 8.

Number	Service	Arguments	Result	Tier	Closest RARS ecall (a7)
`0x00`	EXIT	r0 = status	Exits with status r0 & 0xFF	Core	93 Exit2
`0x10`	PUTCHAR	r0 = character	None	Core	11 PrintChar
`0x11`	PUTS	r0 = address of a NUL-terminated string	None; no newline added	Core	4 PrintString
`0x12`	PUTINT	r0 = signed integer	None; decimal, no newline	Core	1 PrintInt
`0x13`	PUTHEX	r0 = value	None; `0x` and 8 uppercase hex digits	Core	34 PrintIntHex
`0x14`	GETCHAR	None	r0 = next byte of stdin, or -1 at end of file	Core	12 ReadChar
`0x15`	GETINT	None	r0 = next decimal integer on stdin, skipping white space; 0 if none	Core	5 ReadInt
`0x22`	READ	r0 = descriptor, r1 = buffer, r2 = count	r0 = bytes read, 0 at end of file, or -9	Core	63 Read
`0x23`	WRITE	r0 = descriptor, r1 = buffer, r2 = count	r0 = bytes written, or -9	Core	64 Write
`0x30`	INSTRET	None	r0, r1 = instructions retired, counting this SVC	Core	None; RISC-V reads its instret counter
`0x31`	CYCLES	None	r0, r1 = cycles, counting this SVC	Extended	None; RISC-V reads its cycle counter
`0x32`	RAND	None	r0 = next xorshift32 value	Extended	41 RandInt
`0x40`	DUMP_REGS	None	None; labelled registers and NZCV to stderr (format below)	Core	None
`0x41`	DUMP_MEM	r0 = address, r1 = byte count	None; address-prefixed hex bytes to stderr (format below)	Extended	None
`0x42`	TRACE_ON	None	None; resumes the trace	Extended	None
`0x43`	TRACE_OFF	None	None; suspends the trace	Extended	None

RAND is Marsaglia's xorshift32, so every simulator produces the same numbers: `x ^= x << 13; x ^= x >> 17; x ^= x << 5;` then return x. The state starts at `--seed`, or 2463534242.

GETINT skips ASCII whitespace, consumes an optional sign and decimal digits, and returns the low 32 bits of the signed value. If there are no digits, it returns 0 and leaves the non-whitespace input, including any sign, unread. PUTCHAR writes the low byte of r0; PUTS writes the bytes before the NUL terminator. Do not re-encode those bytes as Unicode characters.

READ validates and writes only the bytes actually obtained from stdin, up to the requested count; at end of file it returns 0 without accessing the buffer. WRITE validates all requested bytes before writing them. A zero-byte transfer accesses no memory. Invalid descriptors return -9 before buffer validation.

DUMP_REGS reports r0 to r12, sp, lr, pc and NZCV to stderr, with pc equal to the address of the SVC. Label each register value, for example r0=0000002a, and the flags as NZCV=0010. Values use eight hexadecimal digits, optionally prefixed by 0x. Case, whitespace, line wrapping, register order, r13/r14/r15 aliases and a colon instead of an equals sign are accepted.

DUMP_MEM writes address-prefixed rows of hexadecimal bytes to stderr, for example 20000000: 01 80 ff. Use eight hexadecimal digits for each row address and two for each byte, in increasing address order. Case, whitespace, optional 0x prefixes and the number of bytes per row are flexible. The grader checks the values from both dump services; WRITE to stderr is compared byte for byte in its dedicated test.

The C template provides iss_putchar, iss_puts, iss_putint, iss_puthex, iss_getchar, iss_getint, iss_read and iss_write as direct SVC wrappers. For the other services, add wrappers of your own in the same style below the runtime block; BUILDING.md shows how. Returning from main invokes EXIT. READ and WRITE return raw counts or -9, without errno. Use fixed arrays and your own integer and string-processing code; printf, scanf, allocation and file APIs are not provided.

6. The simulator's interface

Every simulator takes the same command line, reads the same input and writes the same trace, because one grader runs them all.

```
iss [--disasm] [--trace FILE] [--max-instr N] [--seed N] [--cpu m0|m0plus] PROGRAM
```

Option	Meaning	Default
`--disasm`	Print a disassembly listing instead of running the program.	Off
`--trace FILE`	Write the instruction trace to FILE.	No trace
`--max-instr N`	Stop after N instructions.	100000000
`--seed N`	Starting state for RAND.	2463534242
`--cpu`	Timing used by CYCLES.	`m0`

The program's output goes to stdout and stderr. Your simulator's own messages go to stderr only.

Input format

PROGRAM is a section dump in `readelf -x` format, from Compiler Explorer or the local build helper. Each data line holds an address and up to 16 bytes in memory order, then an ASCII column you ignore:

```
Hex dump of section '.vectors':
  0x00000000 00000420 01010000                    ... ....

Hex dump of section '.text':
  0x00000100 094811df 0a2000f0 03f812df 002000df .H... ....... ..
```

Read the lines that start with `0x` after their leading spaces, skip the rest, and put each byte at its address. A byte outside ROM and RAM is a load error. You may also accept a raw binary, loaded at address 0, when the file name ends in `.bin`.

Disassembly listing

CP1 tests your decoder before it executes anything, with objdump as the answer key; use Compiler Explorer's disassembly pane or the local helper's .objdump file. With `--disasm`, print one line per instruction in the dump's `.text` section and exit with status 0:

```
00000100: 4809      ldr r0, [pc, #36]
00000102: df11      svc 17
00000104: 200a      movs r0, #10
00000106: f000f803  bl 110
```

We compare your disassembly with objdump's at every address where objdump shows an instruction, ignoring spacing, case, `.n` and `.w` suffixes, `<label>` names and `@` comments. Everything else must match objdump's style: `svc 17` in decimal, branch targets as bare hex, and register lists written out, as in `push {r4, r5, r6, r7, lr}`. Lines objdump shows as `.word` or `.short`, such as literal pools and padding, are skipped.

Trace format

With `--trace`, write one line per instruction executed:

```
<pc> <encoding> <changes> NZCV=<flags> ; <anything you like>
```

- pc is 8 lowercase hex digits; the encoding is 4 hex digits, or 8 for BL with the first halfword first.
- The changes are the registers whose value changed, in the order r0 to r12, sp, lr, as `r4=0000000a`. pc is left out; the next line shows it.
- Then every instruction memory write in address order, as [address]=value with 8, 4 or 2 hex digits for a word, halfword or byte.
- The flags are always printed as four binary digits. The grader ignores everything after the semicolon, so put your disassembly there. A faulting instruction gets no line; EXIT gets a line if tracing is enabled. TRACE_ON includes its own line when --trace is present; TRACE_OFF excludes its own line. Memory written by a service such as READ is not listed. TRACE_ON does not create a trace when --trace was omitted.

The start of the factorial program from section 7:

```
00000100 4809 r0=0000012c NZCV=0000 ; ldr r0, [pc, #36]
00000102 df11 NZCV=0000 ; svc #0x11
00000104 200a r0=0000000a NZCV=0000 ; movs r0, #10
00000106 f000f803 lr=0000010b NZCV=0000 ; bl 0x110
00000110 b510 sp=2003fff8 [2003fff8]=00000000 [2003fffc]=0000010b NZCV=0000 ; push {r4, lr}
00000112 0004 r4=0000000a NZCV=0000 ; movs r4, r0
```

Exit status

Status	Meaning
0 to 252	The program called EXIT with this status.
253	The program could not be loaded.
254	The instruction limit was reached; print `LIMIT at pc=0x%08x` to stderr.
255	A fault, with the message from section 3.

Test programs exit with 0 to 127, so these never collide. An EXIT with status 253, 254 or 255 also exits with that status, so do not use those values.

7. Building programs

Use the tested templates linked below and the source files in templates/. Both the assembly and bare-metal C templates produce linked, executable dumps in Compiler Explorer. The comparison link produces unlinked objects for instruction inspection and size measurement only. Keep the supplied startup code, vector table and build flags unchanged. The grading inputs are the supplied prebuilt dumps.

Thumb assembly with runnable dump: https://godbolt.org/z/e8asPbvj7

Bare-metal C with runnable dump: https://godbolt.org/z/bGc7P8YEf

Thumb and RV32I comparison: https://godbolt.org/z/jYG4v53bn

For C, copy c_template.c and write main using its direct SVC wrappers. Compile with -mcpu=cortex-m0 -mthumb -O2 -ffreestanding -fno-builtin -fno-stack-protector -nostdlib and the template's fixed section addresses. Division, floating point and some complex C operations may require compiler helpers: avoid them or implement them yourself. Do not add libraries. See BUILDING.md for the wrapper API and optional offline helper.

1. For assembly, open the Thumb template and keep its first comment containing main(void) when replacing the program. That comment prevents Compiler Explorer from inserting a C main stub.
2. Wait for the readelf pane to update. The assembly and C templates show linked sections. The comparison template shows an unlinked object, which is not an executable dump.
3. Copy the complete assembly or C readelf output into a file such as fact.dump. The optional local helper produces the same dump format.
4. Run `iss fact.dump`, and paste the same dump into `refiss` to see what the answer should be.

The factorial program behind the trace in section 6:

```
        @ main(void) is not used: entry is _start.
        .syntax unified
        .cpu    cortex-m0
        .thumb

        .section .vectors, "a"
        .word   0x20040000        @ initial sp: top of RAM
        .word   _start            @ reset vector

        .text
        .thumb_func
        .global _start
_start: ldr     r0, =msg
        svc     #0x11             @ PUTS
        movs    r0, #10
        bl      fact
        svc     #0x12             @ PUTINT
        movs    r0, #0
        svc     #0x00             @ EXIT

        .thumb_func
fact:   push    {r4, lr}          @ r0 = n, returns n!
        movs    r4, r0
        cmp     r0, #1
        ble     1f
        subs    r0, #1
        bl      fact
        muls    r0, r4, r0
        pop     {r4, pc}
1:      movs    r0, #1
        pop     {r4, pc}

        .section .rodata
msg:    .asciz  "10! = "
```

It prints `10! = 3628800` and exits with status 0 after 85 instructions.

In assembly, put .thumb_func before every label whose address you take, so the assembler sets bit 0, and load addresses and large constants with ldr rX, =value, adding .ltorg if a function grows long. The template's -mcpu=cortex-m0 rejects anything that is not an M0 instruction, but not the M0 instructions we left out. C target programs use direct SVC wrappers for input and output.

Using refiss

refiss is one HTML file, refiss-v1.1.html, in the project folder on Canvas. Download it and open it in Chrome, Edge, Firefox or Safari by double-clicking it. Nothing is installed and nothing is uploaded: the page runs entirely in your browser, and it works offline once you have it.

1. Paste your dump into the Program box, or load the `.dump` file with the Open button.
2. Type anything the program reads from stdin into the Input box, and set `--seed` or `--cpu` in the options row if your program needs them.
3. Press Run. The page shows stdout, stderr, the exit status and the trace, in the format of section 6.
4. To debug, paste your own simulator's trace into the Compare box. The page shows the first line where the two traces differ, ignoring everything after the semicolons. Download trace saves the reference trace as a file.

In the browser, `refiss` stops after 10 million instructions; the grader allows 100 million. The version number is at the top of the page. When we fix something we post a new version on Canvas and announce it, so make sure you are using the latest.

8. Extensions

Each extension is one group of the Extended services. Every team does both. Show each at the interview with a program of your own.

Extension	Services	Show it with
Cycle counting	CYCLES, using the Cortex-M0 timings, or the M0+ timings under `--cpu m0plus`	The CPI of your four comparison programs on both cores
Debugging aids	TRACE_ON, TRACE_OFF, DUMP_MEM, RAND	A program that fills an array with RAND, dumps it, sorts it with only the sort traced, and dumps it again

Use this course cycle model on both cores: start with one cycle per retired instruction, add one per data-memory access (one per register transferred by LDM, STM, PUSH or POP), add a taken-branch penalty of 2 for M0 or 1 for M0+, and add one more for BL. Instruction fetches do not count as data accesses. A taken branch incurs its penalty even when its target equals the next instruction. This includes taken conditional B, unconditional B, BL, BX, BLX, POP with pc, and ADD or MOV to pc.

Assume zero memory wait states and the single-cycle multiplier. SVC takes one cycle regardless of its service; service memory accesses add no cycles. INSTRET and CYCLES include their own SVC. A faulting instruction adds neither a retired instruction nor cycles. The model is a deterministic course convention, informed by the Technical Reference Manuals in section 11, rather than a full hardware timing simulation.

9. Comparing Thumb with RV32I

This is the part of the project that is really about RISC-V: you measure the same four programs on both instruction sets and explain the differences with the instructions themselves.

Team measurements

The programs are string length, bubble sort of 20 words, recursive Fibonacci of 15, and the sum of sixteen 64-bit integers. Write each by hand in Thumb and in RV32I, reusing your RARS homework where you can, and once in C. Compile the C for both architectures at -O2 -ffreestanding -fno-builtin. Use identical input data for all versions and include it in your report. For each version, record:

- code size: the .text size of an object containing only the measured function and its own helpers, excluding runtime, main and I/O. Use readelf -SW on an object built with the same compiler and flags as the code you execute; see BUILDING.md;
- instructions executed: count from the first instruction of the measured function through its return, including nested calls. Use a trace or corrected INSTRET deltas on your simulator and RARS instruction stepping/statistics for RV32I. Exclude the caller, startup, I/O and measurement instructions;
- CPI on both cores; cycle counting is required.

The comparison template produces unlinked object code for size measurement, not a runnable dump. To run a Thumb version, put the function and a caller in the course assembly or C runtime template. To run an RV32I version, place the hand-written function or GCC assembly in a RARS harness with the same inputs. Translate GNU assembler directives as needed without changing the executable instruction sequence, resolve any helper calls, and verify the result before measuring.

For INSTRET or CYCLES sampled before and after a call, subtract the caller and measurement overhead from both deltas, including the second counter-read SVC; list that overhead in your report. CPI is the corrected cycle count divided by the corrected instruction count. For RARS, count only the same function interval and explain how you excluded the harness.

Hand in one table and, in at most two pages, explain the three largest differences, with the Thumb and RV32I sequences side by side. A claim with no instructions or numbers behind it earns nothing.

Individual reflection

Each member writes one page alone, answering four of these seven questions with examples from their own code and data.

1. A 64-bit add is ADDS then ADCS on the M0 but four instructions on RV32I. What does a flags register buy, and what does it cost a pipelined or out-of-order core?
2. Most 16-bit Thumb instructions reach only r0 to r7. Find a spill or an extra move in compiled code that this caused.
3. Find three places where you missed x0, and show what you wrote instead.
4. Compare loading 0x12345678 from a literal pool with `lui` and `addi`, in bytes, instructions and memory traffic.
5. Compare PUSH and POP with a run of `sw` and `lw` in a prologue and epilogue. Which is smaller, and which is faster on a simple five-stage pipeline?
6. Your decoder handles about twenty formats; RV32I has six. Sketch both decoders in hardware and argue which is cheaper.
7. Take an array loop from compiled code. Which of the M0's addressing modes does it use, and what does RV32I do instead?

We compare the reflections within a team; they should not read alike.

10. Testing, grading and the interview

Most of the grade comes from automated tests, but the interview decides how much of it each of you keeps.

Tests

Each checkpoint has a public test folder of dumps with their expected stdout, exit status, trace and any checked stderr/debug output, and a script that runs them all on your simulator. Run it before every tag. CP3 and the final checkpoint are also graded on hidden tests.

You also write at least twelve small test programs, at least four by each member, in `tests/` with their expected output and exit status, aimed at the corners: `0xFFFFFFFF + 1` and its carry, signed overflow on ADDS and SUBS, register shifts by 0, 31, 32, 33 and 255, LDM with the base in the list, a call through a function pointer with BLX, and every fault in section 3. Check them against `refiss`, and tell us if `refiss` and the manual disagree.

After CP3 we also run every team's tests against every other team's simulator; tests that expose real bugs earn bonus marks (see Bonus below).

Grading

Part	Weight
CP1	10%
CP2	20%
CP3	25%
Extensions	15%
Comparison study: team measurements	10%
Comparison study: individual reflection	10%
Your tests and the quality of your code	10%

Bonus

Bonus marks add up to 20% on top of the project grade: up to 10% for the team, shared by every member, and up to 10% for each member on their own. The interview factor multiplies bonus marks too, so a bonus you cannot explain is worth nothing.

Bonus	Who	Up to
Bug hunting: after CP3, each of your tests that exposes a real bug in another team's simulator earns 1%.	Team	4%
Exceptions: SysTick and the M0 exception model (vector table entries, stacking and EXC_RETURN), shown with an interrupt-driven program of your own.	Team	4%
A big program: at least 300 lines of C, built with the C template, that runs correctly on your simulator and on `refiss`.	Team	2%
Beat the compiler: a hand-written Thumb version of a function we announce at CP2 that executes fewer instructions than GCC's `-O2` code on our input earns 2%, and 2% more if it is also smaller.	Individual	4%
Find our mistakes: each confirmed error in `refiss` or in this handout earns 1%, for the first person to report it.	Individual	3%
Interview challenge: finish REV16 or REVSH early and add the other one too.	Individual	3%

Interview

In the week after the final deadline each team meets us for about 20 minutes, and every member answers alone. Expect to explain any part of the code we choose, decode an instruction from its hex and show where your decoder handles it, add REV16 or REVSH while we watch, and answer questions on your reflection. The interview gives each of you a factor between 0 and 1 that multiplies your project grade. Working code does not protect a member who cannot explain it.

11. Rules and references

The simulator must be your team's own work, written from the manuals. You may use the Arm manuals below, objdump output, Compiler Explorer and `refiss` as a black box, and you may read existing simulators such as thumbulator, naming any that influenced you in your README. You may not build any part of your simulator on an existing emulator or simulator, by linking, wrapping or translating it; that includes rp2040js (the engine inside `refiss`), Unicorn, QEMU and thumbulator. Code is not shared between teams.

You may use AI tools to learn: to explain an instruction, a passage of the manual, a compiler message or a concept you are stuck on. You may not use them to generate code, for your simulator or your test programs. The code you submit is yours, and you must be able to explain every line of it. If you cannot explain a single line, it is not yours, and submitting it is cheating. At the interview, we choose the lines.

Late tags follow the late policy in the syllabus. The manuals are free from developer.arm.com; search for the document number.

Document	Number	Use it for
ARMv6-M Architecture Reference Manual	DDI 0419	The specification: every encoding and its exact pseudocode
Cortex-M0 Devices Generic User Guide	DUI 0497	A readable summary of each instruction
Cortex-M0 Technical Reference Manual	DDI 0432	Instruction timings for the cycle model
Cortex-M0+ Technical Reference Manual	DDI 0484	M0+ timings for the cycle model

12. Thumb for RV32I programmers

A quick translation from what you know in RARS. Where the two columns look alike, check the manual anyway.

Topic	RV32I	Cortex-M0 (Thumb)
Instruction size	32 bits (16 with the C extension)	16 bits, except BL at 32
Registers	x0 to x31; x0 always reads 0	r0 to r12, sp, lr, pc; no zero register; most 16-bit instructions reach only r0 to r7
Register roles	a0 to a7 arguments, s0 to s11 saved, t0 to t6 temporaries, ra, sp	r0 to r3 arguments and scratch, r4 to r11 saved, r12 scratch, lr, sp
Program counter	Not in the register file; read it with `auipc`	r15; reads as the current address plus 4; writing it is a branch
Condition flags	None	N, Z, C, V in the APSR
Compare and branch	`blt a0, a1, label`	`cmp r0, r1` then `blt label`
Set on less than	`slt`, `sltu`	None; branch on the flags instead
Small constant	`li a0, 5` (`addi a0, x0, 5`)	`movs r0, #5`, for 0 to 255
Large constant	`lui` then `addi`	`ldr r0, =value`, from a literal pool
Copy a register	`mv a0, a1` (`addi a0, a1, 0`)	`movs r0, r1`, or `mov r0, r1` to leave the flags alone
Call	`jal ra, f`	`bl f`

Topic	RV32I	Cortex-M0 (Thumb)
Return	`ret` (`jalr x0, 0(ra)`)	`bx lr`, or `pop {..., pc}`
Arguments and result	a0 to a7, then the stack; result in a0 (and a1)	r0 to r3, then the stack; result in r0 (and r1)
Saving registers	`addi sp, sp, -16` and one `sw` per register	`push {r4-r7, lr}`: one instruction
Addressing modes	One: base register plus a signed 12-bit offset	Five, listed in section 4
Multiply and divide	`mul`, `div`, `rem` in the M extension	`muls` in the base set; no divide, so the compiler calls a library routine
64-bit add	`add`, `sltu`, `add`, `add`	`adds`, `adcs`
System call	`ecall`, service number in a7	`svc #n`, number inside the instruction
Reset	Starts at a fixed reset address; software sets sp	Loads sp and pc from the vector table at address 0

13. Starting points

The skeletons `iss.c` and `iss.py` on Canvas do the same thing; keep the one in your language. Each loads a dump, sets up the memory map, performs the reset and runs the fetch loop with the instruction limit. It raises the memory faults and decodes three instructions, MOVS with an immediate, unconditional B and SVC, plus EXIT, PUTCHAR, PUTS and PUTINT. Everything else is marked TODO.

```
gcc -O2 -Wall -o iss iss.c && ./iss prog.dump
python3 iss.py prog.dump
```

On the factorial program from section 7, either one stops with `FAULT UNDEFINED at pc=0x00000100`, the first line of your to-do list.